K23γ: Software Development for Algorithmic Problems

Winter semester 2023-24

1st Programming Assignment

Vector search and clustering in C/C++

The assignment will be implemented on a Linux system and will be submitted to the Assignments of e-class at the latest on Friday 27/10 at 23.59.

Assignment description

A. Implement the LSH algorithm for vectors in the *d*-dimensional space based on the Euclidean metric (L2), as well as the random projection algorithm on the hypercube for the same metric. The program will be implemented so that taking as input a vector q and integers N and R, it returns approximately: a) the nearest neighbor of q, b) the N nearest neighbors of q and c) the vectors within radius R of q (neighborhood search, range search). The design of the code should allow its easy extension to vector spaces with another metric, e.g., p-norm, or different spaces.

B. Implementation of algorithms for the clustering of vectors in the space $\Re^d$. The L2 metric will be used. The initialization is performed with the k-Means++ technique and the update based on the MacQueen method. The algorithms differ in the assignment step where one of the following algorithms is used: (a) the exact Lloyd's algorithm, or the reverse search through range search with (b) LSH, or (c) Random projection.

INPUT

A.

1) A binary file `input.dat` for the input of the dataset with the following format:

```
[offset] [type]          [value]          [description]
0000     32 bit integer  0x00000803(2051) magic number
0004     32 bit integer  60000            number of images
0008     32 bit integer  28               number of rows
0012     32 bit integer  28               number of columns
0016     unsigned byte   ??               pixel
0017     unsigned byte   ??               pixel
........
xxxx     unsigned byte   ??               pixel
```

This file corresponds to the MNIST dataset which contains images of handwritten numerical digits http://yann.lecun.com/exdb/mnist/. The images are 28*28 pixels, each of which takes an integer value from 0 to 255. The vector that corresponds to each image results from the concatenation of its rows and has dimension 784.

2) Binary file `query.dat` which includes the search set and contains at least one MNIST image with the same format.

The program initially asks the user for the path of the dataset. After the creation of the search structure, the program asks the user for the path of the search file and of the output file. After the execution of the algorithm and the production of the results, the program asks the user if they want to terminate the program or to repeat the search for a different set / search file.

For LSH, the following parameters can be given optionally on the command line: integer number k of the LSH functions hi that are used for the definition of the g, the integer number L of the hash tables, the integer number N of the nearest neighbors and the decimal number R of the search radius. If k, L are not given, the program uses default values k=4, L=5, N=1, R=10000.

For the random projection on the hypercube, the following optional parameters can be given on the command line: the dimension into which the points are projected *k (=d')*, the maximum allowed number of candidate points that will be checked *M*, the maximum allowed number of vertices of the hypercube that will be checked (probes), the integer number N of the nearest neighbors and the decimal number R of the search radius. The default values are: *k*=14, *M*=10, probes=2, N=1, R=10000.

The input and search files can also be given through parameters on the command line.
So the execution will be done through the command:

```
$./lsh –d <input file> –q <query file> –k <int> -L <int> -o <output file> -N <number of nearest> -R <radius>
$./cube –d <input file> –q <query file> –k <int> -M <int> -probes <int> -o <output file> -N <number of nearest> -R <radius>
```

B.
1) The file of case A. 1

2) A parameter configuration file `cluster.conf` with the following format (lines where there is a default value may not be given so the default value is used):

```
number_of_clusters: <int>                   // K of K-medians
number_of_vector_hash_tables: <int>         // default: L=3
number_of_vector_hash_functions: <int>      // k of LSH for vectors, default: 4
max_number_M_hypercube: <int>               // M of Hypercube, default: 10
number_of_hypercube_dimensions: <int>       // k of Hypercube, default: 3
number_of_probes: <int>                     // probes of Hypercube, default: 2
```

The files `input.dat, cluster.conf` are given through parameters on the command line. The execution will be done through the command:

```
$./cluster –i <input file> –c <configuration file> -o <output file> -complete <optional> -m <method: Classic OR LSH or Hypercube>
```

OUTPUT

A. Text file which includes for each image of the search set with the use of the appropriate labels: a) the number of the N-th approximately nearest neighbor that was found and its distance from *q*, b) the distance of *q* from the truly N-th nearest neighbor (through exhaustive search), c) the time of finding (a), d) the time of finding (b) and e) the neighbors within radius R. The output file mandatorily follows the following template:

```
Query: image_number_in_query_set
Nearest neighbor-1: image_number_in_data_set
distanceLSH: <double> [or distanceHypercube respectively]
distanceTrue: <double>
...
Nearest neighbor-N: image_number_in_data_set
distanceLSH: <double> [or distanceHypercube respectively]
distanceTrue: <double>
tLSH: <double>
tTrue: <double>
R-near neighbors:
image_number_A
image_number_B
. . .
image_number_Z
```

and so on. The image_number is determined based on the order in which the image appears in the respective files.

B. A text file which includes the clusters of the images that were produced by each variation of the algorithm, the execution time in each case as well as the internal evaluation index of the clustering **Silhouette**.

The output file mandatorily follows the template below, which is repeated for each variation:

```
Algorithm: Lloyds OR Range Search LSH OR Range Search Hypercube
CLUSTER-1 {size: <int>, centroid: table with the coordinates of the centroid}
.     .     .     .     .     .     .
CLUSTER-K {size: <int>, centroid: table with the coordinates of the centroid}
clustering_time: <double> //in seconds
Silhouette: [s1,...,si,...,sK, stotal]
/* si=average s(p) of points in cluster i, stotal=average s(p) of points in
dataset */

/* Optionally with command line parameter –complete */
CLUSTER-1 {centroid, image_numberA, ..., image_numberX}
.     .     .     .     .     .     .

CLUSTER-K {centroid, image_numberR, ..., image_numberZ}
```

Additional requirements

1) The program must be well organized with separation of the declarations / definitions of the functions, the structures and the data types into logical groups which correspond to separate header and source code files. The quality of the code is also graded (e.g. avoidance of memory leaks). The compilation of the program must be done with the use of the make tool and the existence of an appropriate Makefile.

2) The deliverable must be adequately documented with full commenting of the code and the existence of a readme file which includes at minimum: a) title and description of the program, b) list of the code / header files and their description, c) compilation instructions of the program, d) usage instructions of the program and e) full details of the students who developed it.

3) The implementation of the program must be done with the use of a software version management and collaboration system (Git).